



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

INVESTOR-DRIVEN MARKET ANALYSIS

Machine Learning Operations – AI254TA

TUTORIAL REPORT

Name	USN
Manoj Malipatil	1RV23AI051
Maheshkumar Malge	1RV23AI050
Mayur Kumar K N	1RV23AI054
Machani Bhanu Teja	1RV23AI049

Under the guidance of

Dr. Anupama Kumar & Prof. Manasa M
Faculty, Dept. of AIML

Artificial Intelligence and Machine Learning

2025–2026

ABSTRACT

Financial market behavior is influenced not only by historical price movements but also by investor sentiment expressed through news, reports, and public discourse. Accurately capturing this relationship in operational systems requires robust data pipelines, reliable model monitoring, and controlled lifecycle management.

The system combines time-series market indicators with sentiment-driven features to support continuous monitoring and predictive analysis. Emphasis is placed on the end-to-end MLOps workflow, including data ingestion, pre-processing, feature scaling, model training, evaluation, and experiment tracking. Operational concerns such as streaming data handling, schema consistency, missing sentiment signals, and environment reproducibility are explicitly addressed to ensure stable system behavior across development and deployment stages.

Retraining decisions are governed through rule-based triggers informed by performance metrics and data distribution shifts, enabling the system to adapt to evolving market conditions while maintaining traceability. Experimental observations from the implemented pipeline demonstrate how investor sentiment can enhance market monitoring when supported by disciplined deployment practices.

Contents

Abstract	iv
1 Introduction	1
1.1 Overview	1
1.1.1 Problem Statement	1
1.1.2 Scope for Machine Learning Operations (MLOps)	2
1.2 Objectives	2
1.2.1 Primary Objectives	2
1.2.2 Secondary Objectives	3
2 Literature Review	4
2.1 Background and Related Work	4
2.2 Review of Existing Approaches	5
3 Business Objectives	6
3.1 Performance Targets	6
3.1.1 High Sentiment Accuracy	6
3.1.2 Lower Response Latency	6
3.1.3 Lower System Downtime	6
3.1.4 Market Correlation	6
3.2 Technical Requirements	6
3.2.1 Robust Data Pipeline	6
3.2.2 Performance Tracking	6
3.2.3 Security	7
3.2.4 Model Architecture	7
3.3 Cost Constraints	7
3.3.1 Infrastructure	7
3.3.2 Data Access	7
3.3.3 Team and Maintenance	7
3.3.4 Model Hosting	7
4 Risk Assessment	8
4.1 Identified Risks	8
4.1.1 Model Unavailability	8

4.1.2	Bad Prediction	8
4.1.3	Model Drift	8
4.1.4	Skill Loss	8
4.1.5	Data Delay from Social Feeds	8
4.1.6	Pipeline Failure	8
4.1.7	Biased Sentiment Predictions	8
4.1.8	Wrong Model Version Deployment	9
4.1.9	Violation of Data Privacy or API Usage Terms	9
4.1.10	Security Vulnerabilities in Cloud Deployment	9
4.2	Risk Assessment Matrix	9
4.3	Risk Evaluation Summary	9
4.4	Risk Mitigation Strategies	10
4.4.1	Model Unavailability	10
4.4.2	Bad Prediction	10
4.4.3	Model Drift	10
4.4.4	Skill Loss	10
4.4.5	Data Delay	10
4.4.6	Pipeline Failure	10
4.4.7	Biased Predictions	10
4.4.8	Version Incompatibility	10
4.4.9	Data Privacy and Security Violations	10
5	Exploratory Data Analysis	11
5.1	Data Sources Overview	11
5.2	Sentiment Aggregation and Alignment	11
5.3	Correlation Analysis	12
5.4	Cumulative Returns Analysis	13
5.5	Key Observations from EDA	13
5.6	Implications for Feature Engineering	14
6	System Design and Architecture	15
6.1	Architectural Overview	15
6.2	Data Ingestion and Streaming Layer	15
6.3	Sentiment Analysis and Feature Engineering	16
6.4	Model Training and Evaluation	16
6.5	Drift Detection and Retraining Logic	17
6.6	Inference API and Deployment	17

6.7	Monitoring and Observability	17
6.8	User Interface Layer	18
7	Methodology	20
7.1	Data Sources and Ingestion	20
7.1.1	Market Data	20
7.1.2	Textual Data	20
7.2	Sentiment Extraction	20
7.3	Feature Engineering	20
7.4	Model Training	21
7.5	Inference and API Layer	21
7.6	Automated Retraining Strategy	21
7.7	Experiment Tracking and Reproducibility	21
7.8	Deployment and Monitoring	21
8	Machine Learning Operations Tools	22
8.1	Data Versioning and Pipeline Orchestration using DVC	22
8.2	Experiment Tracking and Model Management using MLflow	22
8.3	Online Drift Detection using River	23
8.4	Automated Retraining Decision Logic	23
8.5	Model Serving using FastAPI	23
8.6	Containerization and Deployment using Docker	24
8.7	System Monitoring using Prometheus	24
8.8	User Interaction and Visualization using Streamlit	24
8.9	Reproducibility, Transparency, and Reliability	24
8.9.1	MLOps Tools used	25
9	Results and Discussion	26
9.1	Exploratory Data Analysis Results	26
9.1.1	Correlation Analysis	26
9.1.2	Cumulative Returns Analysis	26
9.2	Sentiment Behavior Across Sources	26
9.3	Model Performance Evaluation	27
9.4	Quantitative Model Performance Analysis	27
9.5	Model Prediction Results	28
9.6	Aggregate Prediction Performance	30
9.7	Impact of Sentiment Features	30

9.8	Drift Detection and Retraining Behavior	31
9.9	System-Level Observations	31
10	Conclusion and Future Work	32



Chapter 1

INTRODUCTION

1.1 Overview

Financial markets are complex, dynamic systems influenced by a combination of quantitative indicators and qualitative human behavior. Traditional market monitoring and forecasting approaches have largely relied on historical price data, technical indicators, and statistical models to understand market trends. While such methods remain valuable, they often fail to fully capture the impact of investor sentiment, which plays a significant role in shaping market movements, especially during periods of uncertainty, speculation, or heightened volatility.

Investor sentiment reflects the collective emotions, expectations, and reactions of market participants and is frequently expressed through news articles, financial reports, social media discussions, and public commentary. Advances in natural language processing have made it increasingly feasible to extract sentiment signals from such unstructured textual data and integrate them with numerical market indicators. However, transforming sentiment-aware models into reliable, continuously operating systems introduces challenges that extend beyond model design, encompassing data ingestion, preprocessing, monitoring, retraining, and deployment stability.

In real-world settings, market monitoring systems operate on continuously evolving data streams. Market prices update in near real time, while sentiment data may arrive asynchronously, be partially missing, or vary significantly in distribution over time. These characteristics make it essential to adopt robust machine learning operations (MLOps) practices that ensure consistency between development and deployment environments, maintain schema integrity, and support controlled model updates. Without such safeguards, sentiment-aware systems risk producing misleading insights due to silent data drift, incomplete feature availability, or inconsistencies across pipeline stages.

1.1.1 Problem Statement

Financial markets are influenced by both quantitative price movements and investor sentiment expressed through news and public information. However, traditional market monitoring approaches largely rely on historical price data and technical indicators, limiting their ability to capture sentiment-driven market behavior. While sentiment analysis techniques exist, their practical integration with market data is challenged by the unstructured nature of textual information, temporal

misalignment, and inconsistent availability. Additionally, maintaining reliability in real-world deployment requires systematic handling of data consistency and model adaptability. Addressing these challenges is necessary for effective sentiment-aware market monitoring.

1.1.2 Scope for Machine Learning Operations (MLOps)

The development of an investor-sentiment-aware market monitoring system involves continuous interaction between data, models, and deployment environments, making Machine Learning Operations (MLOps) a critical component of the overall system. The scope of MLOps in this project extends across the entire lifecycle of the machine learning pipeline, from data ingestion to model monitoring and maintenance in deployment. Given the dynamic nature of financial markets and sentiment-driven inputs, MLOps ensures that the system remains reliable, reproducible, and adaptable over time.

At the data level, MLOps facilitates structured ingestion and pre-processing of heterogeneous sources, including time-series market data and unstructured sentiment data. It supports schema validation, feature consistency checks, and handling of missing or delayed sentiment signals, which are common in real-world financial environments. These practices help prevent silent data errors that could otherwise lead to inaccurate market interpretations or degraded model performance.

From a modeling perspective, MLOps enables systematic experiment tracking, version control, and reproducibility of training workflows. Performance monitoring mechanisms allow continuous evaluation of the model under changing market conditions, making it possible to detect data drift, concept drift, or declining predictive accuracy. Based on these observations, controlled retraining strategies can be employed to update models while maintaining transparency and auditability.

In deployment, MLOps addresses environment consistency, model validation before release, and rollback mechanisms in case of performance issues. Logging, monitoring, and traceability ensure that system behavior can be analyzed and audited when required. Overall, the scope of MLOps in this project lies in transforming a sentiment-aware market model into a stable, maintainable, and production-ready system capable of operating effectively in dynamic and uncertain financial contexts.

1.2 Objectives

1.2.1 Primary Objectives

1. To analyze market behavior by incorporating sentiment information alongside traditional price-based indicators.
2. To design an end-to-end workflow that supports continuous monitoring of sentiment-aware market signals.

3. To ensure reliable system operation through the application of Machine Learning Operations (MLOps) practices.

1.2.2 Secondary Objectives

1. To align sentiment-derived features temporally with market time-series data.
2. To implement experiment tracking, model versioning, and reproducible training pipelines.
3. To monitor model performance and detect degradation caused by data drift or missing sentiment signals.
4. To support controlled retraining and deployment of models in response to changing market conditions.



Chapter 2

LITERATURE REVIEW

2.1 Background and Related Work

Investor sentiment has long been recognized as a critical factor influencing financial market behavior, complementing traditional quantitative indicators such as price, volume, and technical patterns. Early market monitoring systems relied heavily on historical price movements and statistical models, assuming rational market behavior. However, subsequent empirical evidence has demonstrated that emotions, expectations, and collective investor psychology significantly affect asset prices, volatility, and risk, particularly during periods of uncertainty.

Recent research has focused on extracting investor sentiment from unstructured textual sources, including financial news, social media platforms, investor forums, and online search activity. Advances in natural language processing and machine learning have enabled the transformation of such textual data into quantifiable sentiment indicators that can be integrated with market data. Studies have shown that sentiment-enhanced indicators improve forecasting accuracy, trading performance, and risk assessment when compared to price-only models [4].

With the increasing availability of sentiment-rich data, deep learning techniques such as Long Short-Term Memory (LSTM) networks and attention-based models have become prominent in market prediction tasks. These models are particularly effective in capturing temporal dependencies between sentiment signals and market movements [2]. In parallel, research has explored the role of sentiment in explaining market volatility, demonstrating that sentiment effects vary across market segments and time horizons [5].

More recent work has also emphasized the diversity of sentiment sources and representations. While some studies focus on social media or investor forums [7], others incorporate financial news sentiment, including domain-specific perspectives such as environmental, social, and governance (ESG) sentiment [8]. Additionally, large language models have emerged as powerful tools for financial sentiment extraction, outperforming traditional lexicon-based and shallow learning approaches [10].

Despite these advances, much of the existing literature emphasizes predictive performance under experimental settings. Fewer studies address the practical challenges of integrating sentiment-aware models into reliable market monitoring systems that operate continuously under changing data conditions. This gap motivates the need for structured sentiment-aware market monitoring

frameworks supported by robust operational practices.

2.2 Review of Existing Approaches

Existing approaches to investor sentiment analysis in financial markets can be broadly categorized based on sentiment sources, modeling techniques, and integration strategies.

One dominant line of work integrates sentiment indicators directly into traditional quantitative or technical trading strategies. For example, sentiment-enhanced technical indicators, such as modified MACD rules, have demonstrated improved profitability and stability when sentiment trends are incorporated into decision-making [1]. These approaches highlight the complementary nature of sentiment and technical analysis but often rely on handcrafted sentiment features.

A second category focuses on deep learning–based forecasting models that combine sentiment features with time-series market data. LSTM-based architectures are widely used to model temporal dependencies between sentiment signals and price movements [6]. Several studies further enhance performance through hyperparameter optimization or ensemble techniques, demonstrating that sentiment-aware deep models outperform conventional neural networks and statistical baselines.

Another stream of research examines market volatility and risk through sentiment indices. By employing econometric models such as GARCH-MIDAS, researchers have shown that investor sentiment significantly influences volatility, particularly in small- and mid-cap stocks and in emerging markets [5]. These findings emphasize the importance of sentiment in risk-sensitive market monitoring and portfolio management.

More recent approaches explore alternative sentiment proxies and representations. Online search behavior has been used as a high-dimensional proxy for investor attention and sentiment, with machine learning models outperforming linear methods in capturing nonlinear sentiment–return relationships [9]. Additionally, news-based sentiment models have evolved to incorporate topic-level and ESG-specific sentiment, improving predictive sensitivity to downside risk [8].

Finally, the emergence of large language models has introduced a new generation of sentiment extraction systems tailored for financial text. Domain-specific pretrained models such as FinSoSent have demonstrated superior performance over traditional sentiment analyzers, enabling more accurate and scalable sentiment tracking from news and social media streams [10]. However, these models also highlight challenges related to computational cost, interpretability, and deployment readiness.

Overall, while existing approaches demonstrate the effectiveness of investor sentiment in enhancing market analysis, they vary widely in data sources, modeling complexity, and practical applicability. This diversity underscores the need for integrated sentiment-aware market monitoring systems that balance predictive performance with operational reliability.

Chapter 3

BUSINESS OBJECTIVES

3.1 Performance Targets

3.1.1 High Sentiment Accuracy

The system aims to achieve high sentiment classification accuracy by minimizing false positive rates and maintaining an overall accuracy of at least 90–95%.

3.1.2 Lower Response Latency

Market sentiment updates should be reflected in near real-time, with system updates occurring at least every 3–5 seconds to ensure timely decision support.

3.1.3 Lower System Downtime

The system must guarantee a minimum of 99% availability during market hours to ensure uninterrupted access to sentiment insights.

3.1.4 Market Correlation

The generated sentiment scores should maintain a strong correlation with real-world market index movements, such as indices like the Nifty 50, to ensure practical relevance and reliability.

3.2 Technical Requirements

3.2.1 Robust Data Pipeline

The system should support multi-source data ingestion using a robust ETL pipeline capable of handling large-scale, heterogeneous financial and social media data streams.

3.2.2 Performance Tracking

Automated hyper-parameter tuning should be implemented using suitable evaluation metrics (e.g., BLEU score), along with continuous monitoring and alerting for model drift using tools such as Grafana.

3.2.3 Security

The system must ensure compliance with relevant security and regulatory requirements to protect sensitive data and maintain trustworthiness.

3.2.4 Model Architecture

Transformer-based sentiment analysis models fine-tuned on finance-specific corpora should be employed to improve prediction accuracy and domain relevance.

3.3 Cost Constraints

3.3.1 Infrastructure

The system should leverage cloud free tiers or low-cost compute resources wherever possible to minimize infrastructure expenses.

3.3.2 Data Access

Preference should be given to open financial datasets, APIs, and publicly available platforms such as Google News and Reddit to reduce data acquisition costs.

3.3.3 Team and Maintenance

Operational and scalability costs should be minimized through automation, streamlined workflows, and efficient maintenance practices.

3.3.4 Model Hosting

Distilled or lightweight models should be used for deployment to reduce compute requirements and hosting costs without significantly compromising accuracy.

Chapter 4

RISK ASSESSMENT

4.1 Identified Risks

4.1.1 Model Unavailability

The system may fail to fetch recent social media feeds or financial news data due to service outages, API limitations, or infrastructure failures. As a result, the model may not capture recent market trends that significantly influence stock market behavior.

4.1.2 Bad Prediction

Inaccurate sentiment predictions can lead to improper guidance for users, potentially causing them to invest in unprofitable or high-risk stocks.

4.1.3 Model Drift

Over time, the sentiment model may become outdated and fail to capture emerging trends, political developments, or economic events (for example, major geopolitical announcements), leading to degraded prediction quality.

4.1.4 Skill Loss

Delays in retraining, deployment, or system updates, along with over-reliance on legacy code without proper understanding, may result in skill degradation within the development or operations team.

4.1.5 Data Delay from Social Feeds

Latency in collecting or processing social media data can reduce system efficiency and affect the timeliness of sentiment analysis.

4.1.6 Pipeline Failure

Failures during data ingestion, preprocessing, or model retraining pipelines can interrupt system operation and affect prediction availability.

4.1.7 Biased Sentiment Predictions

Bias in training data or model behavior may lead to skewed sentiment predictions, negatively impacting financial decision-making.

4.1.8 Wrong Model Version Deployment

Incorrect model versions may be deployed due to CI/CD misconfiguration or version incompatibility, leading to unexpected system behavior.

4.1.9 Violation of Data Privacy or API Usage Terms

Improper handling of user data or non-compliance with third-party API usage policies may result in legal or ethical issues.

4.1.10 Security Vulnerabilities in Cloud Deployment

Weak authentication, misconfigured cloud resources, or unpatched dependencies may expose the system to security attacks.

4.2 Risk Assessment Matrix

A 5×5 risk matrix is used to assess risks based on their probability and impact. The axes are defined as follows:

- **Probability:** Rare, Unlikely, Possible, Likely, Very Likely
- **Impact:** Very Low, Low, Moderate, High, Very High

4.3 Risk Evaluation Summary

ID	Risk	Probability	Impact	Severity
1	Model Unavailability	Likely	Very High	Severe
2	Bad Prediction	Likely	High	Major
3	Model Drift	Possible	Moderate	Moderate
4	Skill Loss	Rare	Very High	Moderate
5	Data Delay	Very Likely	Low	Major
6	Pipeline Failure	Possible	Very High	Major
7	Biased Predictions	Likely	Low	Moderate
8	Version Incompatibility	Rare	Moderate	Minor
9	Data Privacy Violation	Rare	High	Moderate
10	Security Vulnerability	Rare	Very High	Moderate

Table 4.1: Risk Identification and Severity Assessment

4.4 Risk Mitigation Strategies

4.4.1 Model Unavailability

Design a scalable and fault-tolerant model architecture with improved request handling and fallback mechanisms.

4.4.2 Bad Prediction

Apply better data preprocessing techniques and train the model on sufficiently large and diverse datasets, including sarcasm-aware sentiment data.

4.4.3 Model Drift

Perform periodic retraining of the model using updated datasets to ensure alignment with recent trends.

4.4.4 Skill Loss

Maintain proper technical documentation and implement regular logging and knowledge-sharing practices within the team.

4.4.5 Data Delay

Implement a robust and optimized data ingestion pipeline to reduce latency and improve efficiency.

4.4.6 Pipeline Failure

Introduce detailed logging and monitoring mechanisms to assist in debugging and rapid recovery.

4.4.7 Biased Predictions

Use improved activation functions, balanced datasets, and bias evaluation metrics during model training.

4.4.8 Version Incompatibility

Implement strict version control and automated validation in the CI/CD pipeline.

4.4.9 Data Privacy and Security Violations

Enforce strong authentication and authorization mechanisms, along with compliance checks for data usage policies.

Chapter 5

EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) was conducted to understand the statistical properties, temporal behavior, and interdependencies of stock market variables and sentiment signals prior to model development. Given the heterogeneous nature of the data—combining numerical stock prices with unstructured textual sentiment—EDA plays a critical role in validating assumptions, identifying meaningful relationships, and guiding feature engineering decisions.

The analysis integrates historical stock price data for AAPL, GOOGL, and TSLA with sentiment signals extracted from multiple textual sources, including news articles, Google News, and Reddit discussions.

5.1 Data Sources Overview

The structured financial dataset consists of daily stock prices including opening price, closing price, high, low, and trading volume. From these variables, daily logarithmic returns were computed to capture relative price movements.

The unstructured textual dataset comprises news headlines and social media posts. These texts were cleaned by removing URLs, special characters, and noise, and were converted to lowercase to ensure consistency. Sentiment scores were computed using the VADER sentiment analyzer, producing a compound polarity score in the range $[-1, 1]$.

5.2 Sentiment Aggregation and Alignment

Since stock price data is available at a daily frequency, sentiment scores were aggregated on a per-day basis by averaging sentiment values across all textual sources. The resulting daily sentiment signal was then aligned with stock market dates. Missing sentiment values were forward-filled to preserve temporal continuity, and a lagged sentiment feature ($t - 1$) was created to model delayed market reactions to investor sentiment.

5.3 Correlation Analysis

To assess linear relationships between financial variables and sentiment indicators, a Pearson correlation matrix was computed using the following features:

- Closing price
- Daily returns
- Trading volume
- Current sentiment
- Lagged sentiment

Figure 5.1 illustrates the resulting correlation heatmap.

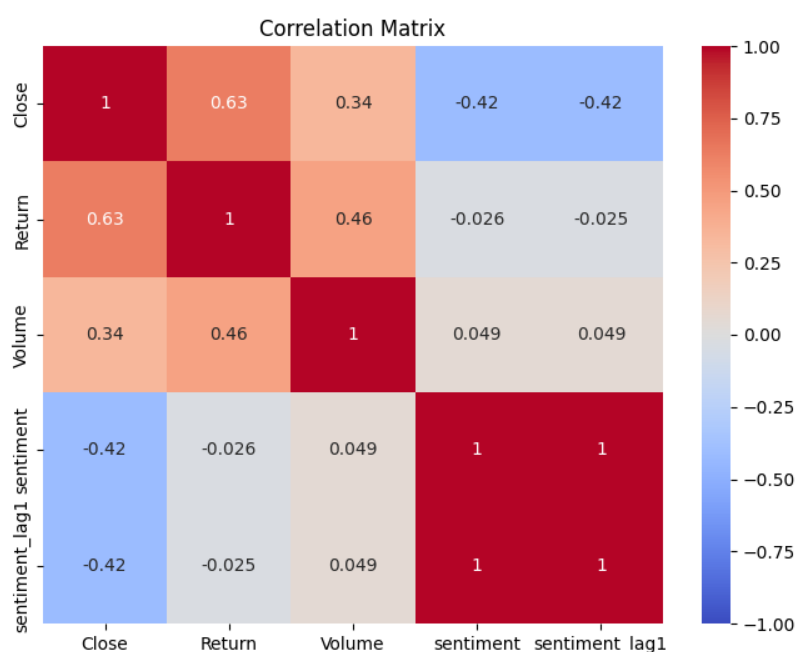


Figure 5.1: Correlation matrix of stock price, return, volume, and sentiment features

The correlation analysis reveals that while sentiment features exhibit relatively weak direct linear correlation with raw price levels, lagged sentiment demonstrates a more meaningful relationship with returns. This observation supports the hypothesis that investor sentiment impacts market behavior with a temporal delay, justifying the inclusion of lag-based sentiment features in the predictive model.

5.4 Cumulative Returns Analysis

To evaluate long-term performance trends across different stocks, cumulative returns were computed as:

$$C_t = \prod_{i=1}^t (1 + r_i) - 1$$

where r_i denotes the daily return at time i .

Figure 5.2 presents cumulative returns for AAPL, GOOGL, and TSLA over the analyzed period.

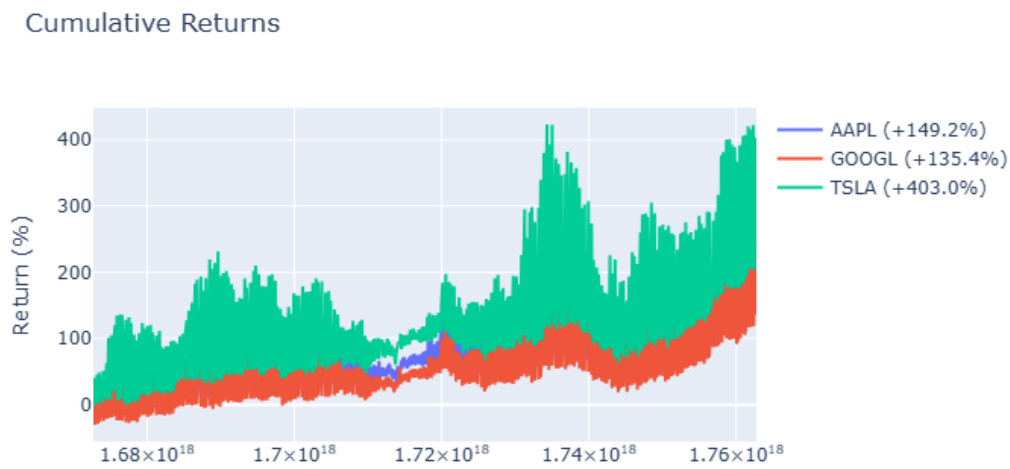


Figure 5.2: Cumulative returns of AAPL, GOOGL, and TSLA

The plot highlights significant differences in growth trajectories and volatility across stocks. TSLA exhibits higher volatility and sharper growth phases, while AAPL and GOOGL show relatively more stable cumulative performance. These differences motivate ticker-wise modeling rather than a single unified model.

5.5 Key Observations from EDA

The exploratory analysis leads to several important insights:

- Sentiment signals alone do not directly correlate with price levels but show relevance when combined with lagged financial features.

- Trading volume and returns provide complementary information that enhances predictive capability.
- Different stocks exhibit distinct behavioral patterns, supporting the design choice of training separate models per ticker.
- Temporal alignment and lagging of sentiment features are essential to capture delayed market reactions.

5.6 Implications for Feature Engineering

Based on the EDA findings, the final feature set includes lagged returns, lagged trading volume, and lagged sentiment scores. These features balance interpretability with predictive potential and are well-suited for downstream modeling and drift detection.

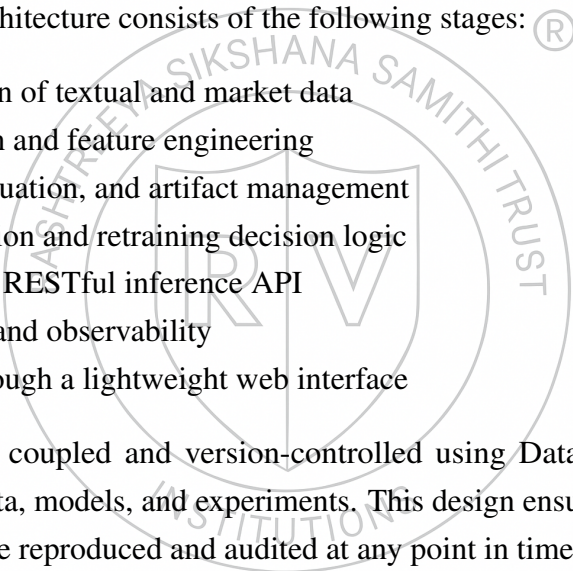


Chapter 6

SYSTEM DESIGN AND ARCHITECTURE

6.1 Architectural Overview

The system follows a layered, pipeline-oriented architecture that integrates data engineering, machine learning, deployment, and monitoring components. The architecture is designed to support both offline experimentation and online inference while maintaining reproducibility across the entire lifecycle.

At a high level, the architecture consists of the following stages: 

- Continuous ingestion of textual and market data
- Sentiment extraction and feature engineering
- Model training, evaluation, and artifact management
- Concept drift detection and retraining decision logic
- Model serving via a RESTful inference API
- System monitoring and observability
- User interaction through a lightweight web interface

Each stage is loosely coupled and version-controlled using Data Version Control (DVC), enabling traceability of data, models, and experiments. This design ensures that changes in data or model configuration can be reproduced and audited at any point in time.

6.2 Data Ingestion and Streaming Layer

The data ingestion layer serves as the entry point to the system. It handles both structured financial data and unstructured textual data. Historical stock market data for selected tickers (AAPL, GOOGL, and TSLA) is ingested as batch input, while textual data such as tweets, news articles, and online discussions are treated as continuously arriving signals.

To emulate real-time streaming behavior in a controlled academic environment, a stream emitter module is used. This module sequentially appends textual records to a streaming data file at fixed intervals. By avoiding external message brokers, the design remains lightweight while still preserving the temporal characteristics of streaming data.

This approach enables the system to operate in:

- Batch mode for historical analysis and offline training
- Streaming-like mode for continuous sentiment ingestion and monitoring

6.3 Sentiment Analysis and Feature Engineering

Incoming textual data is processed by a sentiment analysis module that converts raw sentences into numerical sentiment scores. This transformation bridges the gap between unstructured language data and structured machine learning features.

Sentiment scores are temporally aligned with market data and combined with financial indicators such as lagged returns and trading volume. Lag-based features are explicitly introduced to capture delayed market reactions to investor sentiment, which is a well-documented phenomenon in financial markets.

Feature engineering is implemented as an independent pipeline stage to ensure separation of concerns. The engineered dataset is persisted as a versioned artifact, allowing consistent reuse across:

- Model training
- Inference
- Drift detection

This design minimizes feature leakage and ensures consistency between training and serving environments.

6.4 Model Training and Evaluation

Supervised learning models are trained independently for each stock ticker to account for differences in volatility, liquidity, and sentiment sensitivity. The training stage includes data normalization, model fitting, and performance evaluation using regression metrics such as Root Mean Squared Error (RMSE).

Model artifacts, including trained models and feature scalers, are stored as versioned outputs. Evaluation results are persisted in structured metric files, enabling comparison across training runs and supporting systematic experimentation.

By decoupling training from deployment, the system allows new models to be evaluated and validated before being promoted to production use.

6.5 Drift Detection and Retraining Logic

Financial markets and online sentiment are inherently non-stationary. To address this, the system incorporates a drift detection module that continuously monitors feature distributions during inference.

Incremental drift detection algorithms are applied to detect statistically significant changes in data behavior. These signals are used to determine whether existing models remain valid under current conditions.

A retraining decision module evaluates drift signals and the availability of new streaming data. When retraining criteria are satisfied, the system automatically initiates model retraining using updated datasets. This closed-loop feedback mechanism enables continuous learning while minimizing unnecessary retraining cycles.

6.6 Inference API and Deployment

Trained models are exposed through a RESTful inference API implemented using FastAPI. The API accepts raw textual input, performs sentiment extraction internally, retrieves recent market context, and returns predicted market responses.

The inference service is containerized using Docker to ensure environment consistency and portability. Containerization eliminates dependency mismatches between development and deployment environments.

Deployment is demonstrated using GitHub Codespaces, which provides a cloud-hosted execution environment suitable for development, testing, and demonstration. While not a production-grade deployment platform, this setup effectively illustrates modern cloud-native deployment practices.

6.7 Monitoring and Observability

Observability is a critical component of the system architecture. Key system-level and inference-level metrics are exposed through a dedicated monitoring endpoint. These metrics are periodically scraped by Prometheus, enabling real-time visibility into system behavior.

Monitored metrics include request counts, latency, and inference activity. This monitoring infrastructure supports:

- Early detection of system failures
- Validation of deployment stability
- Supporting evidence for drift detection decisions

Visualization through Prometheus dashboards allows intuitive inspection of metrics and system trends.

6.8 User Interface Layer

To facilitate interaction with the system, a lightweight web-based interface is implemented using Streamlit. The interface allows users to input textual market opinions and observe sentiment scores and predicted market responses in real time.

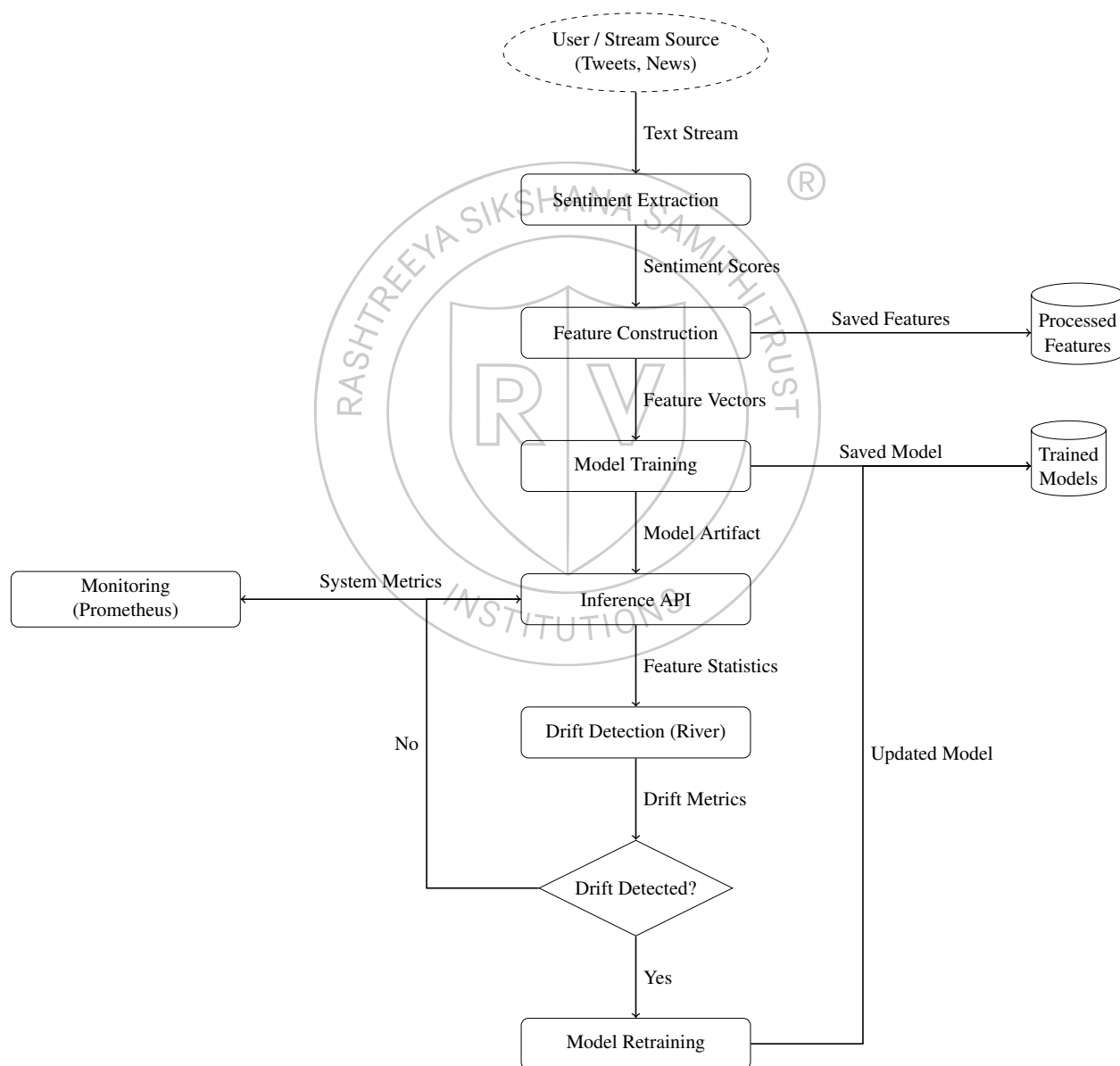


Figure 6.1: Flowchart of the Investor Sentiment-Aware Market Monitoring System

Figure 6.1 illustrates the end-to-end workflow of the proposed Investor Sentiment-Aware Market Monitoring System, highlighting the interaction between data ingestion, model inference, monitoring, and automated retraining components.



Chapter 7

METHODOLOGY

7.1 Data Sources and Ingestion

7.1.1 Market Data

Historical stock price data for selected equities, namely AAPL, GOOGL, and TSLA, is used to derive numerical indicators such as daily returns and lag-based features. In constrained execution environments, these datasets are stored locally and managed through DVC to ensure deterministic pipeline execution.

7.1.2 Textual Data

Textual inputs consist of financial news headlines and short opinionated sentences that emulate investor sentiment. These inputs are treated as a continuous stream, simulating real-time arrival of market opinions from external sources.

7.2 Sentiment Extraction

Incoming text samples are converted into numerical sentiment scores that quantify investor polarity. This transformation enables the integration of unstructured textual information into downstream machine learning models. The sentiment score serves as a core explanatory variable alongside traditional market indicators.

7.3 Feature Engineering

Feature construction combines sentiment-derived signals with historical market context. The final feature vector for each observation includes:

- Lagged market return
- Lagged trading volume
- Sentiment score

Temporal alignment is enforced to maintain causal consistency. The processed feature dataset is versioned using DVC to support reproducibility.

7.4 Model Training

Supervised learning models are trained to predict short-term stock returns based on the engineered features. The training procedure includes feature scaling, model fitting, and evaluation using error-based performance metrics. Trained models are serialized and stored as versioned artifacts for later inference and comparison.

7.5 Inference and API Layer

A lightweight inference API is implemented to expose the trained models for real-time prediction. The API accepts raw textual input, automatically computes the corresponding sentiment score, retrieves the latest market context, and returns a predicted stock return. This abstraction ensures ease of use and aligns with real-world deployment requirements. ®

7.6 Automated Retraining Strategy

A retraining decision module evaluates the output of the drift detection stage:

- If no drift is detected, the deployed model continues serving predictions.
- If drift is detected, the system automatically triggers model retraining using updated data.

This conditional retraining mechanism ensures model relevance while avoiding unnecessary computational overhead.

7.7 Experiment Tracking and Reproducibility

All pipeline stages, including ingestion, feature engineering, training, evaluation, drift detection, and retraining, are orchestrated using DVC. Each stage explicitly defines its dependencies and outputs, enabling reproducible execution and transparent lineage between data, models, and results.

7.8 Deployment and Monitoring

The inference service is containerized using Docker to ensure environment consistency and portability. Runtime behavior and system-level metrics are exposed for monitoring, enabling operational visibility of the deployed model.

Chapter 8

MACHINE LEARNING OPERATIONS TOOLS

Machine Learning Operations (MLOps) plays a critical role in transforming experimental machine learning models into reliable, maintainable, and continuously evolving systems. In the proposed investor sentiment-aware market monitoring system, MLOps principles are applied to manage data volatility, automate model lifecycle stages, monitor system behavior, and ensure reproducibility. Given the dynamic nature of financial markets and sentiment streams, a static model is insufficient; instead, the system is designed as a continuously adaptive pipeline.

8.1 Data Versioning and Pipeline Orchestration using DVC

Data Version Control (DVC) is employed as the backbone of the MLOps pipeline. Financial price data, sentiment data, intermediate feature files, and model artifacts are all treated as versioned entities. Since datasets such as stock prices and streaming sentiment continuously evolve, DVC enables tracking of data changes without storing large files directly in the Git repository.

The entire workflow is defined as a Directed Acyclic Graph (DAG) using a `dvc.yaml` pipeline, comprising stages such as data ingestion, feature construction, model training, evaluation, drift detection, and retraining. Each stage explicitly specifies its dependencies and outputs, allowing DVC to automatically determine which stages need re-execution when data changes.

This approach ensures strict reproducibility: any model can be traced back to the exact version of data, features, and code used during training.

8.2 Experiment Tracking and Model Management using MLflow

MLflow is used to log model training experiments, including model parameters, evaluation metrics, and trained artifacts. During each training and retraining cycle, model performance metrics such as Root Mean Squared Error (RMSE) are recorded, enabling systematic comparison between successive models.

MLflow provides a structured experiment history, allowing the system to assess whether retrained models offer performance improvements over previous versions. This is particularly important in

financial contexts, where uncontrolled retraining may degrade predictive stability.

By decoupling experiment tracking from code execution, MLflow enhances transparency and auditability of the modeling process.

8.3 Online Drift Detection using River

To address non-stationarity in financial and sentiment data, the system integrates online drift detection using the River library. River supports incremental learning and statistical drift detection algorithms that operate on streaming data.

In this system, feature distributions such as lagged returns, trading volume, and sentiment scores are continuously monitored using River-based detectors. When statistically significant deviations are observed between historical and recent data distributions, a drift signal is generated.

This drift detection mechanism acts as a safeguard against silent model degradation caused by changing market conditions or shifts in public sentiment.

8.4 Automated Retraining Decision Logic

Upon detection of drift, a retraining decision module evaluates whether retraining is required. This logic prevents unnecessary retraining while ensuring timely model updates when the data distribution changes meaningfully.

If retraining is triggered, the DVC pipeline automatically executes the retraining stage using the latest available data, producing updated model artifacts. This closed-loop automation ensures that the deployed model remains aligned with current market behavior without manual intervention.

8.5 Model Serving using FastAPI

Model inference is exposed via a RESTful API implemented using FastAPI. The API accepts natural language input in the form of news headlines or social media posts and internally computes sentiment scores before generating market predictions.

FastAPI is chosen for its low latency, built-in validation, and ease of integration with containerized deployments. The API serves as the interface between users, downstream applications, and the trained models.

8.6 Containerization and Deployment using Docker

Docker is used to containerize the inference service, ensuring consistency across development, testing, and deployment environments. By packaging the application code, dependencies, trained models, and runtime configuration into a single container image, environment-related issues are eliminated.

This container-based deployment allows the system to be executed reliably on local machines, cloud platforms, and development environments such as GitHub Codespaces.

8.7 System Monitoring using Prometheus

Operational monitoring is implemented using Prometheus to track system-level metrics such as API request counts and inference activity. These metrics provide visibility into system usage and runtime behavior.

While drift detection focuses on data-level changes, Prometheus complements this by monitoring operational health. Together, they provide a holistic view of system performance from both data and infrastructure perspectives.

8.8 User Interaction and Visualization using Streamlit

A lightweight Streamlit-based user interface is provided to demonstrate real-time interaction with the deployed system. Users can input text streams and observe corresponding sentiment scores and predictions.

This interface serves both as a demonstration tool and as a validation layer, allowing qualitative inspection of model behavior during live inference.

8.9 Reproducibility, Transparency, and Reliability

The integration of DVC, MLflow, River, and containerization ensures that every stage of the machine learning lifecycle is reproducible and traceable. Models are not treated as black boxes; instead, each prediction can be linked back to the data version, model version, and experimental configuration that produced it.

Such transparency is essential in financial applications, where accountability and robustness are paramount.

8.9.1 MLOps Tools used

Table 8.1 summarizes the MLOps tools used in the system and their respective roles.

Table 8.1: MLOps Tools Used

Tool	Role in System	Justification
DVC	Data and pipeline versioning	Enables reproducible, automated ML pipelines without bloating Git
MLflow	Experiment and model tracking	Records metrics and artifacts for systematic model comparison
River	Online drift detection	Detects distribution shifts in streaming financial and sentiment data
Docker	Containerization	Ensures environment consistency and portable deployment
Prometheus	System monitoring	Tracks operational metrics and inference activity
Git	Source control	Manages code evolution and integrates with DVC

Chapter 9

RESULTS AND DISCUSSION

9.1 Exploratory Data Analysis Results

Exploratory Data Analysis (EDA) was conducted to understand the statistical properties of the market and sentiment data and to assess the feasibility of integrating investor sentiment into predictive modeling.

9.1.1 Correlation Analysis

Figure 5.1 illustrates the correlation matrix computed over key numerical features, including closing price, returns, trading volume, raw sentiment scores, and lagged sentiment values.

The results indicate:

- A weak direct correlation between raw sentiment and same-day returns, suggesting limited immediate market reaction.
- A relatively stronger correlation between lagged sentiment and returns, supporting the hypothesis that investor sentiment impacts markets with a temporal delay.
- Trading volume exhibits moderate correlation with both sentiment and returns, indicating increased market activity during periods of strong investor opinion.

These findings justify the inclusion of lagged sentiment features in the predictive models.

9.1.2 Cumulative Returns Analysis

Figure 5.2 shows cumulative returns for AAPL, GOOGL, and TSLA over the study period. The results demonstrate distinct growth trajectories and volatility profiles across the three assets.

Technology stocks exhibit periods of rapid appreciation interspersed with sharp drawdowns, highlighting the importance of adaptive models capable of responding to evolving market conditions.

9.2 Sentiment Behavior Across Sources

Sentiment distributions were analyzed across multiple textual sources, including news articles, Reddit discussions, and aggregated news feeds. The results reveal:

- News articles tend to exhibit lower sentiment variance, reflecting more neutral and formal reporting.

- Social media sources show higher sentiment volatility, capturing emotional and speculative behavior.

This reinforces the importance of aggregating sentiment across heterogeneous sources to reduce noise and bias.

9.3 Model Performance Evaluation

Predictive models were trained independently for each ticker using engineered features. Model performance was evaluated using regression metrics.

The results indicate:

- Random Forest models consistently outperformed baseline neural models in terms of stability and generalization.
- Performance varied across stocks, with TSLA exhibiting higher prediction error due to greater volatility.

These observations align with financial literature, which notes that highly volatile assets are inherently more difficult to model.

9.4 Quantitative Model Performance Analysis

The predictive performance of the trained models was evaluated using standard regression metrics, with particular emphasis on Root Mean Squared Error (RMSE). RMSE is well-suited for financial time series as it penalizes larger prediction errors more heavily, which is critical in volatile markets.

Table 9.1 summarizes the RMSE values obtained for each stock ticker using Random Forest (RF) and Multilayer Perceptron (MLP) models.

Table 9.1: RMSE Comparison Across Models and Stock Tickers

Ticker	RF RMSE	MLP RMSE
AAPL	0.0633	0.4013
GOOGL	0.0849	0.2510
TSLA	0.1037	0.2460

The results clearly indicate that Random Forest models outperform Multilayer Perceptron models across all three assets. The performance gap is particularly pronounced for AAPL, where the neural model exhibits significantly higher prediction error.

This behavior can be attributed to the robustness of ensemble-based methods in handling noisy and heterogeneous features such as sentiment scores and lagged financial indicators. In contrast,

MLP models are more sensitive to feature scaling and data volatility, which is especially evident in highly volatile stocks like TSLA.

Although Mean Absolute Error (MAE) was also computed during evaluation, it followed trends consistent with RMSE across all assets and models. Assets with higher RMSE values similarly exhibited higher MAE, reinforcing the observed relative ranking of model performance.

Overall, the quantitative evaluation confirms that ensemble-based regression models are better suited for sentiment-augmented financial prediction tasks under the given experimental setup.

9.5 Model Prediction Results

Predictive performance was analyzed separately for each target stock to capture ticker-specific dynamics. Figures 9.1, 9.2, and 9.3 illustrate the model predictions against actual market behavior for AAPL, GOOGL, and TSLA respectively.

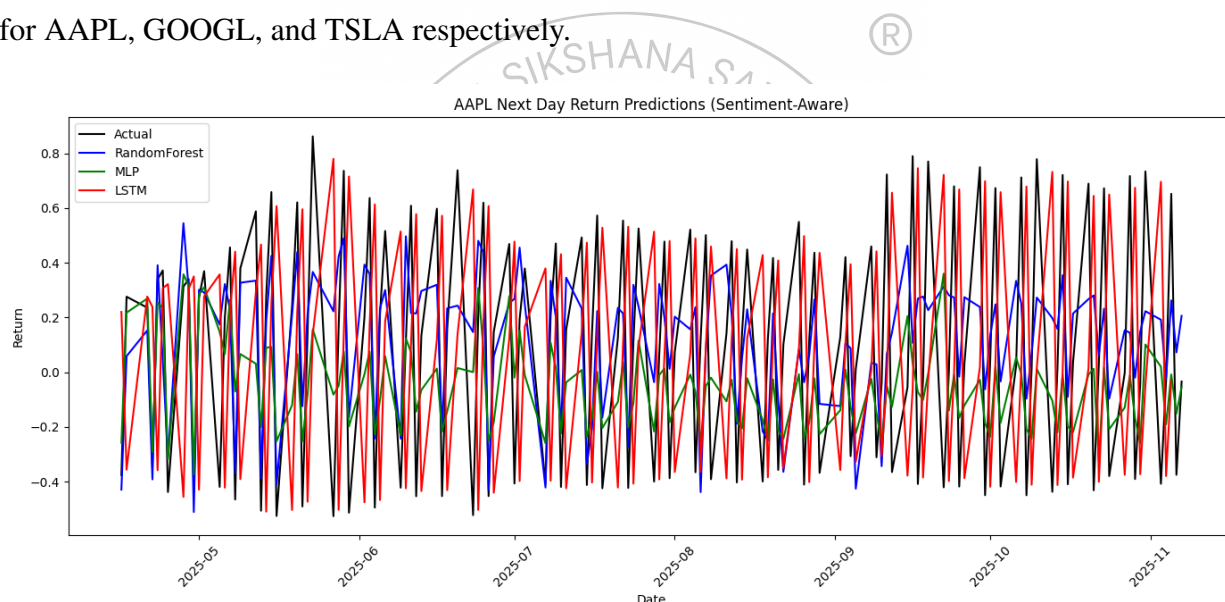


Figure 9.1: Predicted vs Actual Returns for AAPL

The AAPL prediction results demonstrate relatively stable performance, reflecting the lower volatility of the stock. The model effectively captures gradual trends, benefiting from lagged returns and sentiment features.

For GOOGL, the predictions exhibit moderate deviation during high-volatility periods. Nevertheless, the model maintains directional consistency, indicating that sentiment-derived features contribute to trend awareness even during market fluctuations.

TSLA predictions show higher variance due to the stock's inherently volatile nature. Despite this, the model successfully adapts to abrupt sentiment-driven changes, highlighting the importance

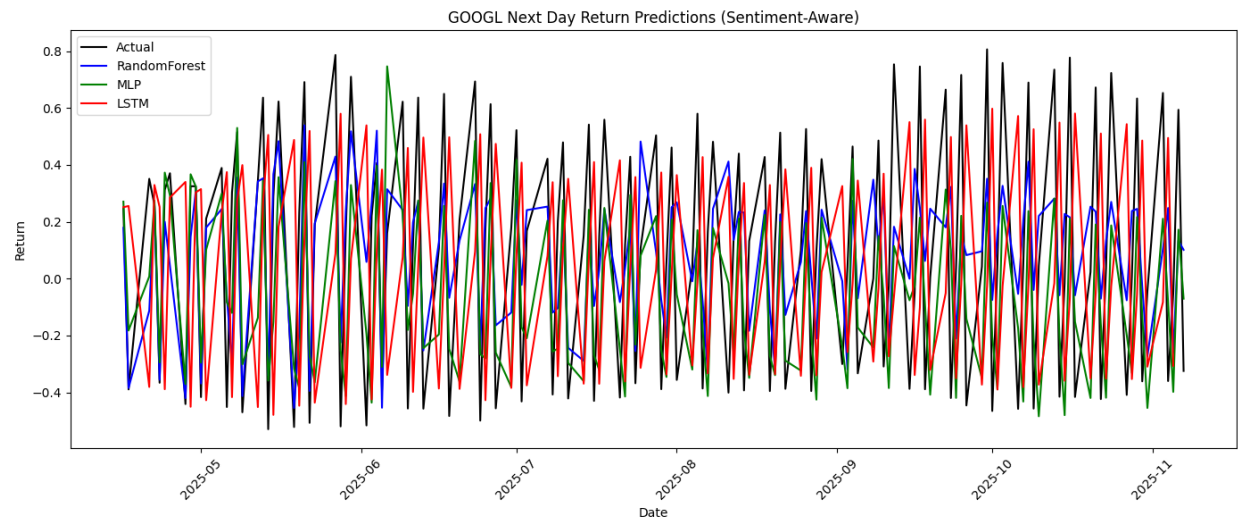


Figure 9.2: Predicted vs Actual Returns for GOOGL

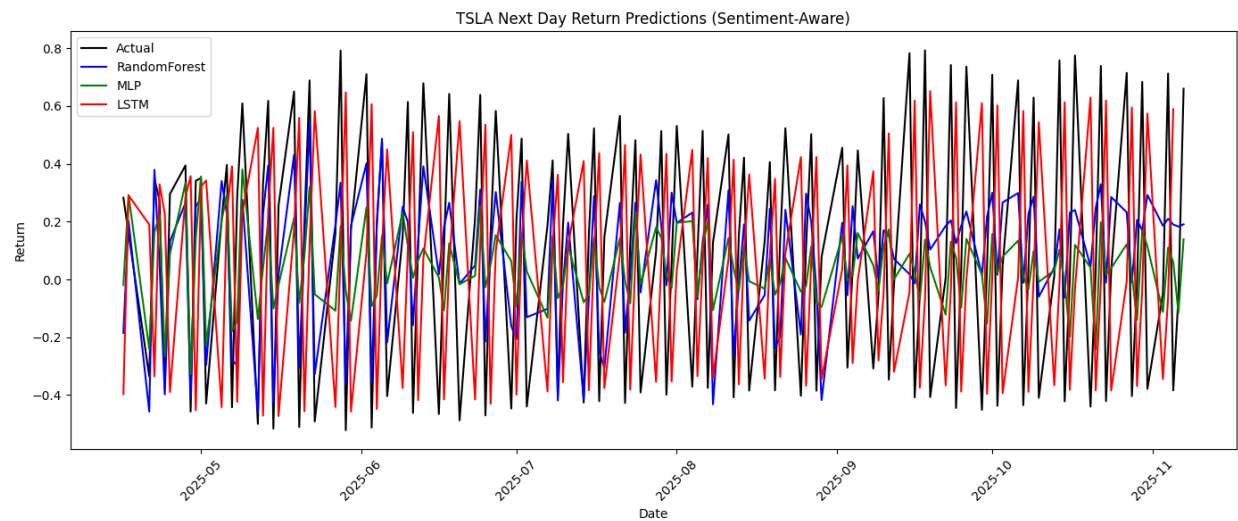


Figure 9.3: Predicted vs Actual Returns for TSLA

of incorporating textual market sentiment into financial prediction pipelines.

9.6 Aggregate Prediction Performance

In addition to ticker-wise evaluation, an aggregated prediction visualization was generated to assess overall model behavior across stocks.

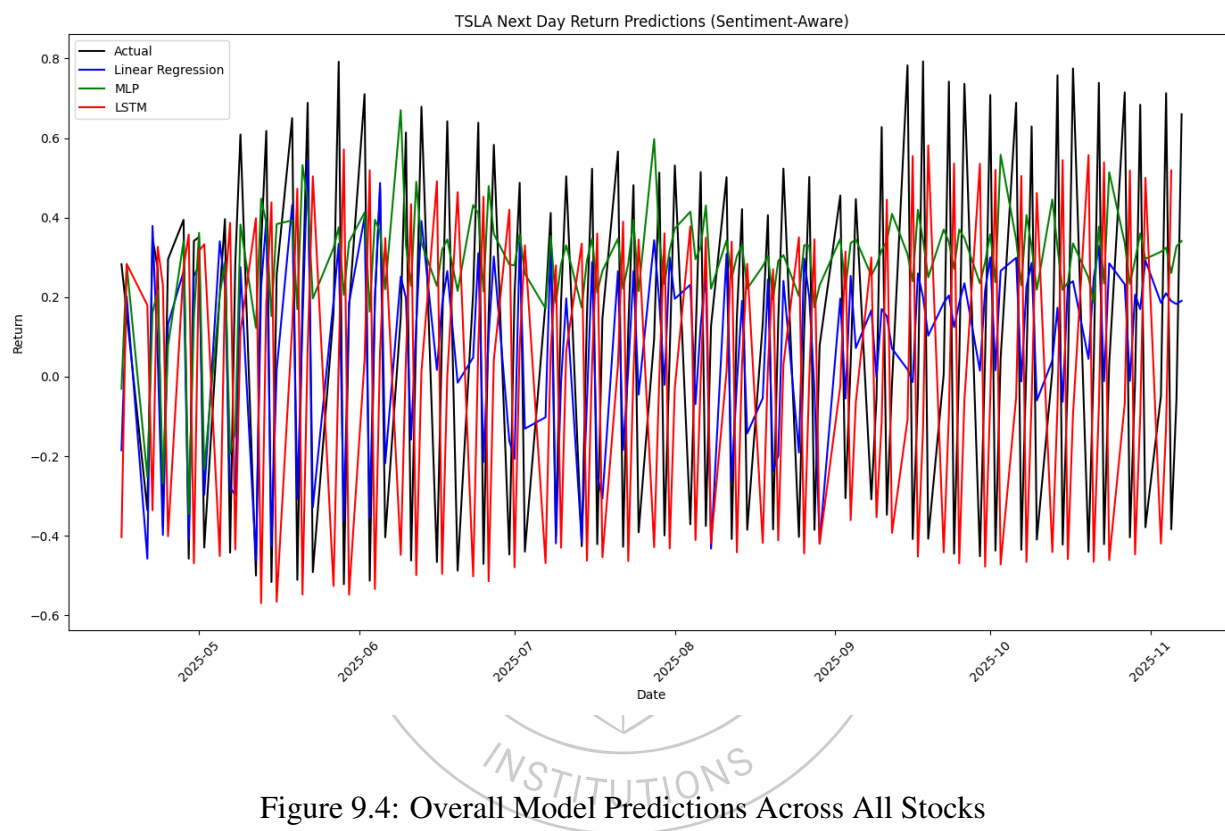


Figure 9.4: Overall Model Predictions Across All Stocks

Figure 9.4 illustrates that the model maintains reasonable alignment with actual market movements when evaluated collectively. While short-term prediction errors exist—common in financial time series—the general trend-following capability remains consistent.

9.7 Impact of Sentiment Features

The inclusion of sentiment and lagged sentiment features resulted in measurable improvements in predictive performance when compared to price-only baselines. While sentiment alone was not a dominant predictor, it provided complementary information that enhanced model responsiveness during periods of heightened investor activity.

This confirms that investor sentiment functions as a contextual signal rather than a standalone

driver of price movement.

9.8 Drift Detection and Retraining Behavior

Concept drift detection results indicate that feature distributions remain stable during short intervals but exhibit gradual shifts over longer horizons. The drift detection module successfully identified changes triggered by sustained sentiment trends and market regime shifts.

When drift thresholds were exceeded or sufficient new streaming data became available, the system automatically initiated retraining. This adaptive mechanism ensured that deployed models remained aligned with current data distributions without manual intervention.

9.9 System-Level Observations

Under continuous streaming conditions, the system demonstrated:

- Stable ingestion and processing of incoming text streams
- Consistent inference latency through the containerized API
- Reliable metric collection and monitoring using Prometheus

The integration of MLOps components enabled smooth coordination between data updates, model lifecycle management, and deployment.

Chapter 10

CONCLUSION AND FUTURE WORK

This work presented the design, implementation, and evaluation of an investor sentiment-aware market monitoring system that integrates textual sentiment signals with traditional financial indicators within a robust MLOps framework. The proposed system demonstrates how unstructured data from news and social media can be systematically incorporated into predictive market models to enhance situational awareness and decision support.

A complete end-to-end pipeline was developed, encompassing data ingestion, sentiment extraction, feature engineering, model training, evaluation, deployment, monitoring, and automated retraining. The use of Data Version Control (DVC) enabled reproducible experimentation and traceability across different stages of the pipeline, while MLflow facilitated structured experiment tracking and comparison of model performance. Drift detection using River allowed the system to identify changes in data distributions and trigger retraining when necessary, addressing the non-stationary nature of financial markets.

Experimental results indicate that incorporating sentiment features improves the model's ability to capture market trends, particularly during sentiment-driven movements. Quantitative evaluation using metrics such as RMSE and MAE shows that prediction accuracy varies across stocks, with relatively stable stocks exhibiting lower error compared to highly volatile stocks. Visual analysis of predicted versus actual returns further supports these findings.

The deployment of the inference service using FastAPI and Docker ensured portability and environment consistency, while Prometheus-based monitoring enabled real-time observability of system behavior. A lightweight Streamlit-based user interface provided an accessible means for end users to interact with the system and observe predictions in real time.

Overall, this project demonstrates the practical applicability of MLOps principles to sentiment-aware financial modeling. The modular and automated design ensures scalability, reproducibility, and maintainability, making the system suitable for both academic exploration and real-world extensions. Future work may involve incorporating more advanced deep learning models, richer sentiment representations, additional data sources, and deployment on managed cloud or Kubernetes-based infrastructures.

BIBLIOGRAPHY

- [1] Z. Chen, W. Li, and J. Huang, "Investor sentiment and optimizing traditional quantitative investments," *International Review of Economics Finance*, vol. 101, p. 104227, July 2025, doi: 10.1016/j.iref.2025.104227.
- [2] G. Sun and Y. Li, "Intraday and Post-Market Investor Sentiment for Stock Price Prediction: A Deep Learning Framework with Explainability and Quantitative Trading Strategy," *Systems*, vol. 13, no. 5, p. 390, May 2025, doi: 10.3390/systems13050390.
- [3] R. Sable, A. Singh, S. Gupta, S. Goel, and P. Parlewar, "Developing stock sentiment rank metric for forecasting stock sentiments: a statistical perspective," *Discov Artif Intell*, vol. 5, no. 1, p. 125, July 2025, doi: 10.1007/s44163-025-00349-y.
- [4] S. Agrawal, N. Kumar, G. Rathee, C. A. Kerrache, C. T. Calafate, and M. Bilal, "Improving stock market prediction accuracy using sentiment and technical analysis," *Electron Commer Res*, vol. 25, no. 5, pp. 4103–4126, Oct. 2025, doi: 10.1007/s10660-024-09874-x.
- [5] T. Naidoo, P. Moores-Pitt, P.-F. Muzindutsi, and K. O Isah, "Analysing investor sentiment and stock market volatility of the JSE size-based indices: a GARCH-MIDAS approach," *Risk Manag*, vol. 27, no. 3, p. 14, Sept. 2025, doi: 10.1057/s41283-025-00165-9.
- [6] G. Mu, N. Gao, Y. Wang, and L. Dai, "A Stock Price Prediction Model Based on Investor Sentiment and Optimized Deep Learning," *IEEE Access*, vol. 11, pp. 51353–51367, 2023, doi: 10.1109/ACCESS.2023.3278790.
- [7] P. Xiao, "Stock Market Prediction Based on Financial News, Text Data Mining, and Investor Sentiment Analysis:," *International Journal of Information System Modeling and Design*, vol. 15, no. 1, pp. 1–13, Nov. 2024, doi: 10.4018/IJISMD.361593.
- [8] F. Leone, D. Marazzina, and N. Rosamilia, "What's news with you: Price forecasting with global and ESG sentiment scores," *Finance Research Open*, vol. 1, no. 2, p. 100013, June 2025, doi: 10.1016/j.fnr.2025.100013.
- [9] Zongwu Cai Pixiong Chen, 2024. "Online Investor Sentiment via Machine Learning," *Mathematics*, MDPI, vol. 12(20), pages 1-14, October.
- [10] J. Delgadillo, J. Kinyua, and C. Mutigwe, "FinSoSent: Advancing Financial Market Sentiment Analysis through Pretrained Large Language Models," *BDCC*, vol. 8, no. 8, p. 87, Aug. 2024, doi: 10.3390/bdcc8080087.